

ED05 - Pruebas de Caja Blanca y Caja Negra

Descripción

En esta tarea trabajaremos con una clase Java `GestorJuegos` que implementa un sistema básico de gestión de inventario para una tienda de videojuegos. La clase dispone de los métodos `main`, `registrarLoteJuegos` y `venderJuego`, además de métodos auxiliares como `mostrarStockJuegos` y `obtenerStockActual`, o métodos de validación, además del método `main` que se encarga de ejecutar el programa.

Código Base

Este es el código que deberás tener en cuenta para resolver la tarea:

```
import java.util.HashMap; import
java.util.Map;

/**
 * Clase principal que se encarga de gestionar los juegos
 */
public class GestorJuegos {
    // Utilizamos un map para almacenar los juegos y su cantidad
    private Map<String, Integer> stockJuegos;
    private final int maxStock = 200; // Stock máximo del total de juegos

    public GestorJuegos() {
        stockJuegos = new HashMap<>();
    }

    public static void main(String[] args) {
        GestorJuegos tienda = new GestorJuegos();

        // Registramos un lote de juegos
        if (tienda.registrarLoteJuegos(new String[] { "ABC123", "ABC124", "ABC125" }, new
int[] { 2, 3, 1 }) < 0) {
            System.out.println("Error al registrar los juegos");
        } else {
            System.out.println("Juegos registrados correctamente");
        }

        if (tienda.registrarLoteJuegos(new String[] { "XYZ123", "ABC125" }, new int[]
{ 2, 3, 1 }) < 0) {
            System.out.println("Error al registrar los juegos"); // Debería imprimir
este mensaje por no ser iguales las longitudes de los arrays
        } else {
            System.out.println("Juegos registrados correctamente");
        }
    }
}
```

```

        if (tienda.registrarLoteJuegos(new String[]{"ABC123", "ABC126", "ABC112"},
new int[]{199, 3, 1}) < 0){
            System.out.println("Error al registrar los juegos"); // Debería
imprimir este mensaje por exceder el stock máximo
        } else {
            System.out.println("Juegos registrados correctamente");
        }

        if (tienda.registrarLoteJuegos(new String[]{"XYZ123", "ABC123", "ABC125"},
new int[]{100, 4, 3}) < 0){
            System.out.println("Error al registrar los juegos"); // Debería
imprimir este mensaje por exceder el stock máximo
        } else {
            System.out.println("Juegos registrados correctamente");
        }

        // Mostramos el stock de juegos
        tienda.mostrarStockJuegos();

        // Vendemos un juego
        int cantidadVendida = tienda.venderJuego("ABC123", 2); // Debería vender 2
juegos
        if (cantidadVendida > 0) {
            System.out.println("Se han vendido " + cantidadVendida + " juegos del
código ABC123");
        }

        tienda.mostrarStockJuegos();

        // Vendemos un juego que no existe
        cantidadVendida = tienda.venderJuego("XYZ264", 2); // Debería mostrar un
mensaje de error
        if (cantidadVendida > 0) {
            System.out.println("Se han vendido " + cantidadVendida + " juegos del
código XYZ123\n");
        } else {
            System.out.println("No se ha podido vender el juego con código
XYZ123\n");
        }

        tienda.mostrarStockJuegos();

        // Vendemos un juego con cantidad insuficiente
        cantidadVendida = tienda.venderJuego("ABC123", 20); // Debería mostrar un
mensaje de error
        if (cantidadVendida > 0) {
            System.out.println("Se han vendido " + cantidadVendida + " juegos del
código ABC123\n");
        } else {
            System.out.println("No se ha podido vender el juego con código
ABC123\n");
        }

```

```

    }

    tienda.mostrarStockJuegos();

    //Vendemos un juego con código ilegal
    cantidadVendida = tienda.venderJuego("ABC1234", 2); // Debería mostrar un
mensaje de error
    if (cantidadVendida > 0) {
        System.out.println("Se han vendido " + cantidadVendida + " juegos del
código ABC123\n");
    } else {
        System.out.println("No se ha podido vender el juego con código
ABC123\n");
    }

    tienda.mostrarStockJuegos();

    //vendemos un juego con cantidad negativa
    cantidadVendida = tienda.venderJuego("ABC123", -2); // Debería mostrar un
mensaje de error
    if (cantidadVendida > 0) {
        System.out.println("Se han vendido " + cantidadVendida + " juegos del
código ABC123\n");
    } else {
        System.out.println("No se ha podido vender el juego con código
ABC123\n");
    }
}

private void mostrarStockJuegos() {
    System.out.println("Stock de juegos:");
    for (Map.Entry<String, Integer> juego : stockJuegos.entrySet())
        { System.out.println(juego.getKey() + ": " + juego.getValue());
        }
}

// Metodo que registra un lote de juegos en el mapa de juegos de la tienda
public int registrarLoteJuegos(String[] codigos, int[] unidadesPorCodigo) {
    int juegosRegistrados = 0;

    //comprobamos que las longitudes de los arrays sean iguales
    if (codigos.length != unidadesPorCodigo.length || codigos.length == 0)
        { return -1;
        }

    //Comprobamos que los valores de cantidad sean positivos y sino, la
sumamos a cantidadTotal
    int cantidadTotal = 0;
    for (int cantidad : unidadesPorCodigo)
        { if (cantidad < 0) {
            return -2;
        }
    }
}

```

```

        } else {
            cantidadTotal += cantidad;
        }
    }

    //comprobamos que no se exceda el stock máximo
    if (obtenerStockActual() + cantidadTotal > maxStock)
    { return -3;
    }

    for (int i = 0; i < codigos.length; i++) {
        if (stockJuegos.containsKey(codigos[i])) {
            stockJuegos.put(codigos[i], stockJuegos.get(codigos[i]) +
unidadesPorCodigo[i]);
        } else {
            stockJuegos.put(codigos[i], unidadesPorCodigo[i]);
        }
        juegosRegistrados += unidadesPorCodigo[i];
    }
    return juegosRegistrados;
}

private int obtenerStockActual()
{ int stockActual = 0;
  for (int cantidad : stockJuegos.values())
  { stockActual += cantidad;
  }
  return stockActual;
}

/**
 * Método que vende una cantidad de juegos de un código específico. Devuelve
la cantidad vendida o un código de error en caso de que no se pueda realizar la
venta.
 * Códigos de error:
 * 0: Error de formato en el código o cantidad no válida
 * -1: El juego no existe
 * -2: No hay suficiente stock para vender la cantidad solicitada
 * @param codigo El código del juego a vender
 * @param cantidad La cantidad de juegos a vender
 * @return La cantidad de juegos vendidos o un código de error en caso de que
no se pueda realizar la venta
 */
public int venderJuego(String codigo, int cantidad)
{ try {
    // Comprobamos el formato del código: ABC123
    validarCodigo(codigo);
    validarCantidad(cantidad);

    if (stockJuegos.containsKey(codigo)) {
        if (stockJuegos.get(codigo) >= cantidad) {

```

```

        stockJuegos.put(codigo, stockJuegos.get(codigo) - cantidad);
        return cantidad;
    } else {
        System.out.println("No hay suficiente stock para el juego con
código " + codigo);
        return -2;
    }
} else {
    System.out.println("No existe el juego con código " + codigo);
    return -1;
}
} catch (IllegalArgumentException e) {
    System.out.println("Error: " + e.getMessage());
}
return 0;
}

private void validarCantidad(int cantidad) throws IllegalArgumentException
{ if (cantidad <= 0) {
    throw new IllegalArgumentException("La cantidad debe ser mayor que
0");
}
}

private void validarCodigo(String codigo) throws IllegalArgumentException
{ if (codigo.length() != 6) {
    throw new IllegalArgumentException("El código debe tener 6
caracteres");
}
if (!codigo.substring(0, 3).matches("[A-Z]+")) {
    throw new IllegalArgumentException("Los primeros 3 caracteres deben
ser letras mayúsculas");
}
if (!codigo.substring(3).matches("[0-9]+")) {
    throw new IllegalArgumentException("Los últimos 3 caracteres deben ser
dígitos");
}
}
}
}

```

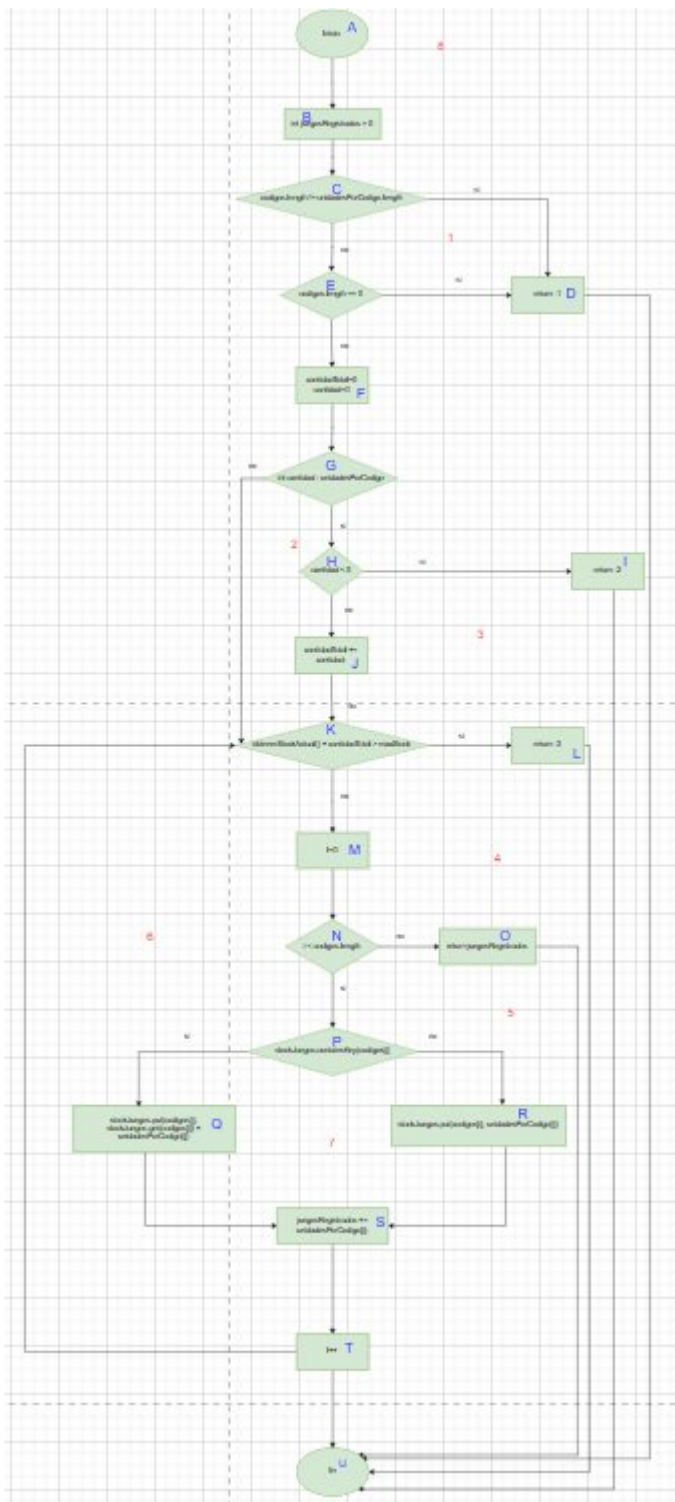
Puedes encontrar el código de esta práctica en el proyecto de IntelliJ IDEA subido a [este enlace](#) de GitHub.

Tareas a Realizar

Deberás crear un documento llamado **apellidos_nombre_ed05.pdf** donde des respuesta a los siguientes apartados:

1. Realiza un análisis de caja blanca completo del método `registrarLoteJuegos`. En este análisis deberás:

- Crear el grafo de flujo del método. Puedes hacerlo con drawio, mermaid o cualquier otra herramienta que prefieras (incluido Paint o, incluso, a mano).



- Calcular la complejidad ciclomática, utilizando las 3 fórmulas disponibles

$$V(G) = A - N + 2 = 21 - 27 + 2 = 8$$

$$V(G) = C + 1 = 7 + 1 = 8$$

$$V(G) = r = 8$$

- Identificar los caminos independientes

A → B → C → D → U

A → B → C → E → D → U

A → B → C → E → F → G → K → L → U

A → B → C → E → F → G → K → M → N → O → U

A → B → C → E → F → G → H → I → U

A → B → C → E → F → G → H → J → G → K → M → N → O → U

A → B → C → E → F → G → K → M → N → P → Q → S → T → N → O → U

A → B → C → E → F → G → K → M → N → P → R → S → T → N → O → U

- Diseñar los casos de prueba para cada camino

Caso de prueba	Entrada	Resultado Esperado	Camino Cubierto
Caso 1	codigos.lenght = 2 unidadesPorCodigo.lenght = 1	return -1	A → B → C → D → U
Caso 2	codigos = [] unidadesPorCodigo = []	return -1	A → B → C → E → D → U
Caso 3	N/A	N/A	A → B → C → E → F → G → K → L → U
Caso 4	codigos = ["J1"] unidadesPorCodigo = [-5]	return -2	A → B → C → E → F → G → K → M → N → O → U
Caso 5	codigos = ["J1"] unidadesPorCodigo = [100000000]	return -3	A → B → C → E → F → G → H → I → U
Caso 6	N/A	N/A	A → B → C → E → F → G → H → J → G → K → M → N → O → U
Caso 7	codigos = ["J1"] unidadesPorCodigo = [10]	return 10	A → B → C → E → F → G → K → M → N → P → Q → S → T → N → O → U
Caso 8	codigos = ["J2"] unidadesPorCodigo = [30]	return 30	A → B → C → E → F → G → K → M → N → P → R → S → T → N → O → U

2. Realiza un análisis de caja negra, incluyendo valores límite y conjetura de errores del método **venderJuego**. Para este análisis considera que:

- El método recibe como parámetro la cantidad de juegos a vender
- La cantidad no podrá ser menor o igual a 0
- La cantidad no podrá ser mayor al stock actual
- El código del juego deberá tener 6 caracteres, los 3 primeros letras mayúsculas y los 3 últimos dígitos
- Al tratarse de pruebas funcionales no es necesario conocer los detalles del código, aunque dispones de él en el código fuente.

Condición de entrada	Clase de equivalencia		Clases válidas	COD	Clases no válidas	COD
Cantidad	Rango	1 a stockActual		V1	< =1	NV1
Cantidad					> stockActual	NV2
Código (Longitud)	Valor exacto	Exactamente 6 caracteres		V2	< 6 caracteres	NV3

Código (Longitud)				> 6 caracteres	NV4
Código (Formato inicio)	Lógica (Tipo de caracteres)	3 Letras Mayúsculas	V3	Contiene minúsculas, números o signos	NV5
Código (Formato fin)	Lógica (Tipo de caracteres)	3 Números (dígitos)	V4	Contiene letras o signos	NV6
Código (Estado)	Lógica (Presente o ausente)	Cadena con texto	V5	Null	NV7

Caso de Prueba	Clase de equivalencia	Entradas	Resultado Esperado
CP_01	V1, V2, V3, V4, V5	codigo = "GTA005" cantidad = 1	Venta procesada con éxito.
CP_02	V1, V2, V3, V4, V5	codigo = "GTA005" cantidad = 10	Venta procesada con éxito.
CP_03	NV1	codigo = "GTA005" cantidad = 0	Error: Cantidad debe ser mayor a 0
CP_04	NV1	codigo = "GTA005" cantidad = -1	Error: Cantidad debe ser mayor a 0
CP_05	NV2	codigo = "GTA005" cantidad = 11	Error: Stock insuficiente.
CP_06	V1, V2, V3, V4, V5	codigo = "COD123" cantidad = 5	Venta procesada con éxito.
CP_07	NV3	codigo = "COD12" cantidad = 5	Error: Formato de código inválido
CP_08	NV4	codigo = "CODI123" cantidad = 5	Error: Formato de código inválido
CP_09	NV5	codigo = "cod123" cantidad = 5	Error: Formato de código inválido
CP_10	NV5	codigo = "CO1123" cantidad = 5	Error: Formato de código inválido
CP_11	NV6	codigo = "COD12A" cantidad = 5	Error: Formato de código inválido

CP_12	NV7	codigo = null cantidad = 5	Error: Formato de código inválido
CP_13	NV4	codigo = "COD 123" cantidad = 5	Error: Formato de código inválido

3. Completa la clase **GestionJuegosTest** del tipo Caso de prueba JUnit en IntelliJ IDEA, que nos permita **pasar las pruebas unitarias de caja blanca del método registrarLoteJuegos**. Los casos de prueba ya los habrás obtenido en el **primer apartado** del ejercicio.

1. Crea un fork del proyecto de GitHub que contiene el código base de esta práctica.
2. Clona tu fork en tu máquina local.
3. Tras completar la clase **GestionJuegosTest** con los casos de prueba de caja blanca, haz commit y push de tus cambios a tu repositorio de GitHub.
4. Añade el archivo **apellidos_nombre_ed05.pdf** con el análisis de caja blanca y caja negra a tu repositorio de GitHub.

Entrega

Incluye el enlace a tu repositorio de GitHub en el formulario de entrega de la práctica. Asegúrate de que tu repositorio es público para poder acceder a él.